Cloud-Native Online Bookstore

DSAA 4040 Final Project Report - E1 Engineering Track

Vue 3 + FastAPI + PostgreSQL + Redis on Kubernetes

Verified with Docker Compose, K3s, Traefik Ingress, HPA, Pod recovery, and PVC persistence

HKUST(GZ) - Spring 2026

DSAA 4040 Course Project - E1 Engineering Track

1. Project Objective and Motivation

This project implements a small but complete cloud-native online bookstore. The goal is to demonstrate the engineering workflow required by E1: application design, containerization, Kubernetes deployment, service exposure, configuration management, and verification of a working system.

The system is intentionally scoped as a course project rather than a production e-commerce platform. It focuses on the end-to-end cloud-native path from a browser request to frontend, backend, cache, database, and Kubernetes resources.

2. System Architecture

The application has four main services.

Component	Technology	Responsibility
Frontend	Vue 3, Vite, Nginx	Bookstore UI and reverse proxy for API traffic
Backend	FastAPI, SQLAlchemy, Uvicorn	REST APIs, business logic, health and metrics endpoints
Database	PostgreSQL 16	Persistent catalog, cart, order, and order item data
Cache	Redis 7	Cached catalog/category reads

Request flow:

```
Browser
-> NodePort or Traefik Ingress
-> Nginx frontend Service
-> /api/* forwarded to FastAPI backend Service
-> PostgreSQL for persistent data
-> Redis for read cache
```

3. Implementation

The frontend supports browsing books, searching/filtering, viewing details, adding books to a cart, checking out, and viewing order history. The backend exposes APIs for health checks, catalog queries, cart operations, order creation, and order listing.

Important implementation details:

- The backend initializes demo catalog data when the database is empty.
- Cart and order workflows use relational tables instead of in-memory state.
- The backend exposes `/api/health` for service health and `/metrics` for simple Prometheus-style counters.
- Catalog and category reads are cached in Redis with `catalog:*` keys and a short TTL.
- The frontend displays live deployment status from the backend health endpoint.
- Nginx serves the SPA and proxies `/api/*` to `backend-service:8000`.

4. Deployment Design

The project includes both Docker Compose and Kubernetes deployment paths.

Docker Compose runs frontend, backend, PostgreSQL, and Redis locally. It was used as the first integration gate before Kubernetes validation.

Kubernetes manifests are under `bookstore/k8s/` and include:

Requirement	Implemented Resource
Containerized app	Dockerfiles for frontend and backend
Deployments	Frontend and backend Deployments with 2 replicas each
Services	NodePort frontend Service, ClusterIP backend/database/cache Services
Configuration	ConfigMap for runtime settings
Secret	Kubernetes Secret for database credentials
Storage	PostgreSQL PVC
Ingress	Traefik-compatible path routing for `/` and `/api`
Robustness	Readiness/liveness probes and resource requests/limits
Scaling	HPA for frontend and backend

The final Kubernetes verification used a single-node K3s cluster with Traefik enabled. This matches the course handout's recommendation to use lightweight Kubernetes environments.

5. Testing and Evaluation

Docker Compose Verification

Docker Compose was built and run successfully. The frontend was reachable at `http://localhost:18080`, and backend health was reachable at `http://localhost:18000/api/health`.

The API workflow was tested with health check, book search, cart insertion, order creation, and order listing.

Kubernetes Verification

The final Kubernetes run verified:

- K3s node ready.
- Traefik and metrics-server running.
- PostgreSQL, Redis, backend, and frontend Pods ready.
- Backend and frontend running with two replicas.
- NodePort endpoint reachable at `http://localhost:30080`.
- Traefik Ingress endpoint reachable at `http://localhost:18081`.

• HPA metrics resolved instead of staying unknown.

• API order workflow worked through Kubernetes routing.

Runtime evidence was collected with:

```
kubectl get pods -n bookstore -o wide
kubectl get svc -n bookstore
kubectl get ingress -n bookstore
kubectl get hpa -n bookstore
kubectl get pods -n kube-system -o wide
kubectl top pods -n bookstore
```

The `kube-system` output showed both Traefik and metrics-server running. This matters because Ingress and HPA were not only declared as YAML files; they were backed by real cluster controllers during the final run.

Autoscaling and Monitoring Evaluation

To test HPA behavior, I added a controlled diagnostics endpoint, `/api/diagnostics/cpu`, which performs bounded CPU work and returns the execution time. This endpoint is not part of the bookstore user workflow; it is used only to create repeatable CPU pressure for autoscaling evaluation.

The HPA experiment used an in-cluster load generator Pod:

```
Target: http://backend-service:8000/api/diagnostics/cpu?iterations=6000000
Duration: 240 seconds
Parallel workers: 14
Backend HPA: min 2, max 8, target 60% CPU
```

Observed result:

Time (UTC)	Backend HPA CPU	Backend Replicas	Interpretation
17:07:10	`2%/60%`	2	baseline before load
17:07:55	`231%/60%`	2	CPU spike detected
17:08:11	`463%/60%`	4	HPA scaled up
17:08:26	`462%/60%`	8	HPA reached max replicas
17:09:12	`231%/60%`	8	load distributed across backend Pods
17:12:16	`2%/60%`	8	load ended; downscale delay observed

During the load, `kubectl top pods` showed backend Pods rising from a few millicores to hundreds of millicores of CPU. After HPA added replicas, the backend workload was spread across eight Pods. The final repository keeps a compact HPA summary and figures rather than the full raw watch logs. This demonstrates the complete cloud-native loop:

```
metrics-server collects CPU
-> HPA observes CPU above target
-> Deployment replica count increases
-> new backend Pods become Ready
-> service continues routing requests
```

The experiment also showed a practical Kubernetes behavior: when the load stopped, CPU dropped quickly, but replicas did not immediately return to 2. This is expected because HPA downscaling is intentionally conservative to avoid oscillation.

For readability and auditability, the raw watch logs were converted into matplotlib figures:

- `deliverables/evidence/visualizations/hpa_autoscaling_timeseries.png`
- `deliverables/evidence/visualizations/backend_cpu_timeseries.png`

These charts make the scaling result easier to audit: the HPA chart shows the replica count changing from 2 to 4 and then 8, while the CPU chart shows backend CPU pressure rising during the load period and then returning to idle.

Database Evidence and Data Visualization

To avoid relying only on API responses, I also queried the PostgreSQL database directly through `psql` inside the cluster:

```
kubectl exec -n bookstore <postgres-pod> -- \
psql -U bookstore -d bookstore -c "select ... from orders ..."
```

The database snapshot showed:

Table	Rows
books	12
cart_items	0
orders	7
order_items	8

The exported CSV files are:

- `deliverables/evidence/db/orders.csv`
- `deliverables/evidence/db/book_inventory_by_category.csv`
- `deliverables/evidence/db/top_sold_books.csv`

The generated matplotlib figure, `deliverables/evidence/visualizations/database_psql_summary.png`, summarizes real order totals and inventory stock by category from PostgreSQL. This confirms that checkout operations produced persistent database records rather than only frontend state.

Redis Cache Evidence

Redis was verified directly from the Kubernetes Redis Pod after catalog API requests:

```
redis-cli keys 'catalog:*'
catalog:categories
catalog:books:q=:category=Cloud Computing
```

The captured evidence in `deliverables/evidence/k8s/redis_cache_evidence.txt` also records TTL and string length for the cached keys. This shows that Redis is active in the read path instead of being only an unused container.

Recovery and Persistence

Frontend and backend Pods were deleted to confirm Deployment recovery. New Pods became ready and the application stayed reachable after recovery.

PostgreSQL was also restarted to test persistence. The database restart caused a short transient backend failure because PostgreSQL is a single Pod, but after recovery the previously created order was still readable. This confirms

PVC persistence while also exposing the limitation of a single-node/single-database setup.

The smoke test also confirmed the application workflow through Kubernetes: health check, book listing, cart insertion, order creation, order listing, and a direct PostgreSQL query for recent orders.

6. Demo Video

The main demo video is stored at `deliverables/video/DSAA4040_E1_bookstore_demo_subtitled.mp4`. It is a screen-recorded walkthrough with burned-in subtitles, based on the Kubernetes NodePort deployment and verified command outputs. The demo covers:

- UI workflow: browse, cart, checkout, order history.
- PostgreSQL order evidence from `psql`.
- Redis `catalog:*` cache evidence.
- HPA scaling from 2 to 8 backend Pods.
- Pod recovery and PVC persistence.

7. Challenges and Fixes

Several issues were encountered during final integration:

Issue	Resolution
Local Kubernetes setup was not already working	Used K3s in Docker with built-in kubectl and Traefik
Ingress needed a real controller	Verified with K3s Traefik instead of only applying manifests
Frontend proxy label/path needed to match backend Service	Updated Nginx and UI labels to `backend-service:8000`
Dependency builds needed to be stable	Pinned backend dependencies and added pip retry options
HPA did not have strong evidence at first	Added a controlled CPU diagnostics endpoint and in-cluster load-test script
K3s-in-Docker deploy script could not read local YAML files	Updated `deploy.sh` to stream manifests into containerized kubectl via stdin
Reusing `latest` images did not restart Pods	Added rollout restart after local image loading
PostgreSQL restart caused temporary API failures	Documented as a limitation and verified recovery plus PVC persistence

8. Limitations and Future Work

The current system meets the E1 engineering requirements, but it is not production grade.

Main limitations:

- PostgreSQL is a single Pod with local persistent storage.

• HPA load testing uses a diagnostics CPU endpoint to create controlled pressure; it does not represent real browsing traffic.

• Secrets are basic Kubernetes `Secret` objects, not integrated with an external secret manager.

• The frontend and backend are simple course-project services without authentication or payment logic.

Future work:

• Add a managed or replicated database.

• Add a user-workload load test against normal browsing/order APIs.

• Add monitoring dashboards with Prometheus and Grafana.

• Add CI/CD deployment automation.

• Add authentication and richer order management.

9. Conclusion

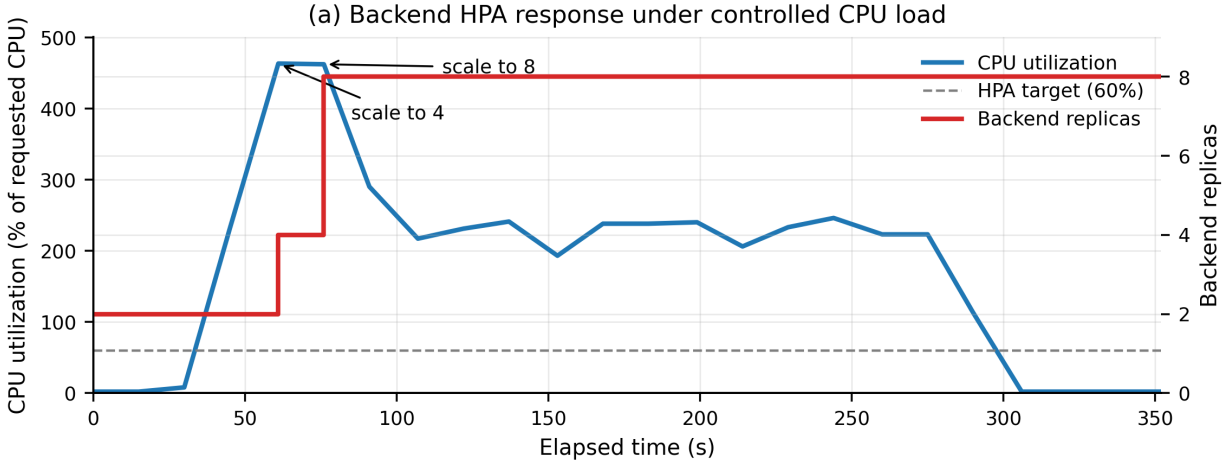
This project delivers a working cloud-native online bookstore and verifies the main E1 requirements: frontend, backend, database, containerization, Kubernetes Deployment and Service, ConfigMap, Secret, Ingress, architecture documentation, and complete deployment workflow.

It also includes several advanced engineering elements such as Redis caching, HPA, health probes, resource limits, metrics endpoint, Pod recovery verification, and PVC persistence testing. The remaining limitations are mainly around production-grade high availability and deeper load evaluation.

10. Tool Usage Disclosure

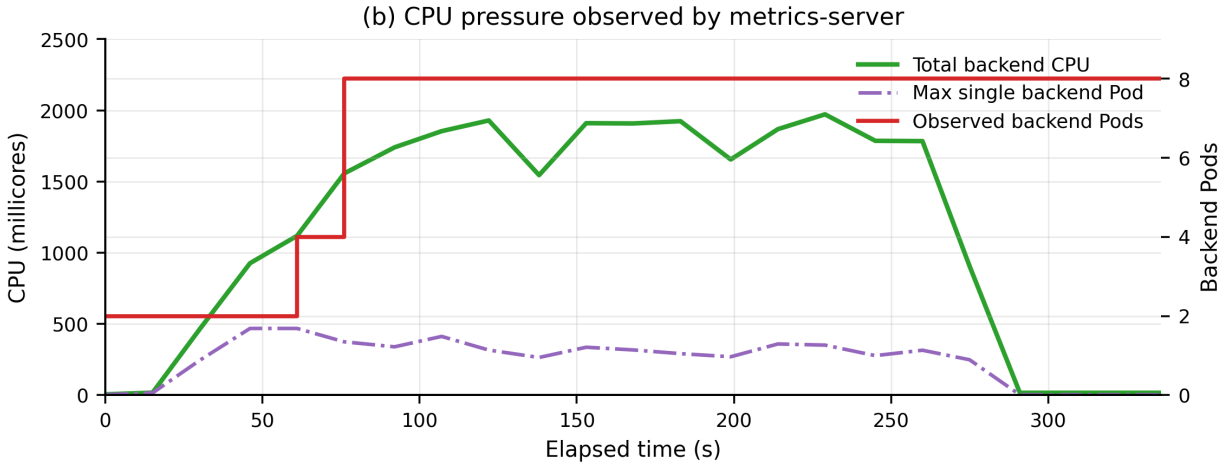
Tools were used to assist debugging and documentation polishing. The system design, implementation choices, Kubernetes validation, experiments, and final verification were completed and reviewed by the team.

Evaluation Results



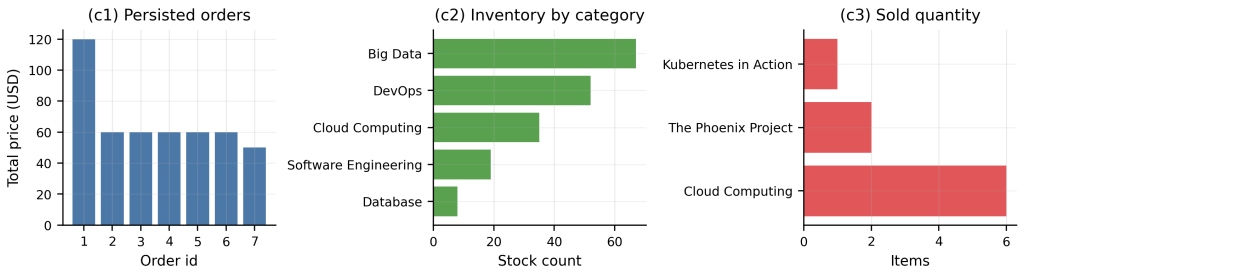
Source: kubectl get hpa sampled every 15s; load generated by an in-cluster curl Pod.

HPA CPU target and backend replica count during controlled load



Source: kubectl top pods sampled every 15s during the same HPA load test.

Backend CPU pressure from kubectl top pods during the HPA test



Source: psql CSV exports from PostgreSQL Pod in namespace bookstore.

PostgreSQL order and inventory evidence exported with psql

Selected UI Evidence